

23-system-runtime

23. System Runtime & Async Programming

Level: □ Intermediate

Jam: 8 (2 sesi)

Prasyarat: Module 1 (JS Fundamentals) — udah pake Promise tapi belum paham dalemnya

Output: Paham gimana JS jalan di komputer — memory, event loop, thread, Promise beneran

□ Sesi 1: Memory, GC, Event Loop, Thread

1.1 Stack vs Heap

Memory komputer itu ada 2 area utama buat nyimpen data:

Stack — memory buat data kecil + sementara.

- Primitives (number, string, boolean, null, undefined)
- Function call frames
- Cepet, otomatis dihapus pas function selesai
- Ukuran terbatas (stack overflow kalo recursion gila-gilaan)

```
function greet(name: string) {  
  const msg = "Halo " + name; // msg di stack  
  return msg;  
}  
// Pas greet selesai, msg otomatis ilang dari stack
```

Heap — memory buat data besar + umur panjang.

- Objects, arrays, functions (semua reference types)
- Lebih lambat dari stack
- Ukuran besar
- Harus dibersihkan manual... atau pake **Garbage Collector**

```
const user = { name: "Budi", age: 17 };
// user (reference) ada di stack
// { name: "Budi", age: 17 } ada di heap
```

1.2 Value vs Reference — Visual

Ini yang paling sering bikin bingung. Bedain:

Primitive (value):

```
let a = 10;
let b = a;    // b = 10, duplikat
b = 20;
console.log(a); // 10 – aman, a ga berubah
```

Object (reference):

```
let obj1 = { count: 10 };
let obj2 = obj1;    // obj2 pointing ke OBJECT YANG SAMA
obj2.count = 20;
console.log(obj1.count); // 20 – obj1 ikut berubah!
```

Visual memory:

Stack:

```
[a: 10]
[b: 20]    ← bebas, punya sendiri
```

Stack:

```
[obj1] ———
[obj2] ———
```



Heap:

```
{ count: 20 }    ← satu object, diubah barengan
```

Imutability pattern:

```
// Biar object ga termutasi, spread aja:
const updated = { ...obj1, count: 20 };
// Sekarang obj1.count masih 10, updated.count = 20
```

1.3 Pointer (Konsep)

JavaScript ga punya pointer explicit kayak C/C++, tapi **semua object pake pointer di belakang layar**.

```
// Reference = pointer yang dibungkus aman
// Ga bisa lo manipulasi alamat memory langsung
// Tapi behavior-nya sama: pointing ke object yang sama
```

```
function updateName(user: { name: string }) {
  user.name = "Budi"; // Ngeubah object asli, bukan copy
}
```

Penting: ini beda bahasa. C/C++ lo bisa `*ptr = 5`. JS ngelindungi lo dari pointer arithmetic — lo ga bisa akses memory sembarangan (makanya lebih aman dari segi security).

1.4 Garbage Collector (GC)

GC = tukang sampah di memory. Ngehapus object yang udah ga dipake.

Mark & Sweep (algoritma dasar V8):

1. MARK: Mulai dari root (global, local variables).
Tandai semua object yang MASIH bisa diakses.
2. SWEEP: Hapus semua object yang GA ditandai.

Setelah sweep, memory yang dibebasin bisa dipake lagi.

Contoh object jadi sampah:

```
function createUser() {
  const user = { name: "Budi" };
  return user;
}

const budi = createUser();
// □ user di dalam function masih dipake, karena budi nunjuk ke situ

budi = null;
// □ Sekarang object { name: "Budi" } GA ADA YANG NUNJUK
// GC bakal hapus ini di siklus berikutnya
```

Memory leak (kebocoran memory): terjadi kalo object GA pernah di-GC karena masih ada reference meskipun udah ga dipake.

```
// Memory leak classic: event listener ga di-remove
class Chat {
  messages: string[] = [];

  constructor() {
    window.addEventListener('message', () => {
      this.messages.push('...');
    });
    // □ GA PERNAH di-remove → Chat object ga bisa di-GC
  }
}
```

```
}
```

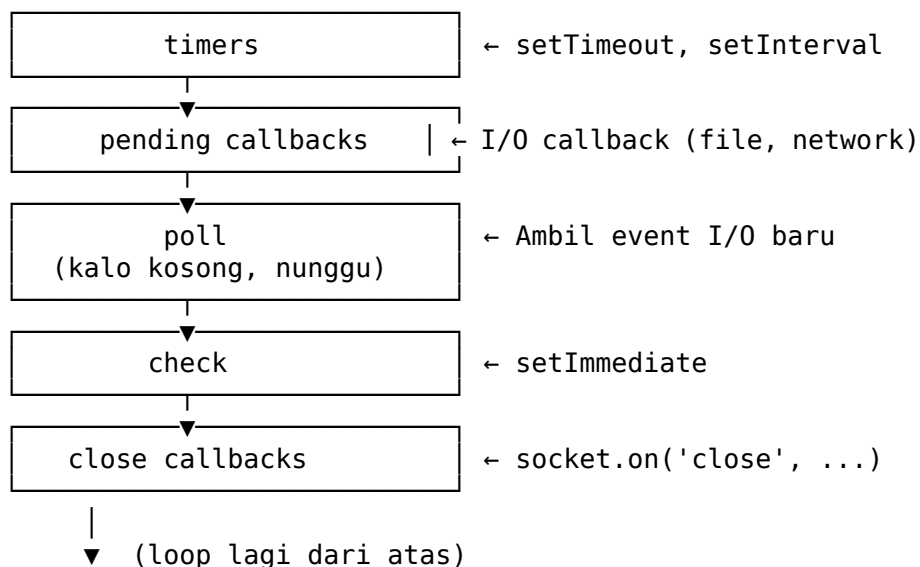
```
// Fix: simpan reference listener, remove pas ga dipake
```

V8 Generational GC:

- **Young Generation** — object baru. Di-scan tiap saat (GC cepet).
- **Old Generation** — object yang selamat dari beberapa siklus GC. Jarang di-scan.
- Kenapa perlu? Biar GC ga nge-scan SEMUA memory tiap saat — performa tetap lancar.

1.5 Event Loop — Jantung Node.js

Node.js itu **single-threaded**... tapi bisa handle ribuan request per detik. Kok bisa? **Event Loop**.



Cara kerja:

1. Jalanin synchronous code dulu (call stack)
2. Kalo ada setTimeout / I/O / Promise, kirim ke sistem
3. Kalo call stack kosong, cek queue
4. Prioritaskan **microtask** (Promise, process.nextTick) DULU
5. Baru **macrotask** (setTimeout, setInterval, I/O)

Contoh:

```
console.log('1'); // Jalan duluan (sync)
```

```
setTimeout(() => {
```

```

    console.log('2'); // Macrotask – jalan paling akhir
  }, 0);

  Promise.resolve().then(() => {
    console.log('3'); // Microtask – jalan sebelum macrotask
  });

  console.log('4'); // Sync

// Output: 1, 4, 3, 2
// Kenapa? Sync → Microtask (Promise) → Macrotask (setTimeout)

```

Kenapa ini penting?

- Lo nulis await bukan berarti "berhenti" — cuma mindahin kode ke microtask queue
- Event loop tetep jalan ngelayanin request LAIN selagi nunggu
- Makanya Node.js bisa handle ribuan request dengan 1 thread

1.6 Single Core vs Multi Thread

Mitos: "JavaScript itu single-threaded." **Fakta:** JavaScript **language** single-threaded. Tapi **runtime** (Node.js, browser) pake banyak thread.

JavaScript main thread:

- Jalanin kode lo
- Event loop
- Call stack

libuv thread pool (4 thread default):

- File system (readFile, writeFile)
- DNS lookup
- Crypto (hash, bcrypt)
- Compression (zlib)

Worker Threads (Node.js 10+):

- CPU-intensive task (image processing, data parsing)
- Lo bikin explicit

CPU-bound vs I/O-bound:

CPU-bound (hitung Fibonacci besar):

- ❑ Jangan di main thread → blockage event loop
- ❑ Pake Worker Thread / pindahkan ke microservice

I/O-bound (read file, query DB, fetch API):

- ❑ Aman di main thread → libuv yang urus

Event loop tetep jalan karena nunggu callback doang

Contoh block event loop:

```
// ❌ INI BAKAL BLOCK EVENT LOOP – semua request lain nunggu
app.get('/heavy', (req, res) => {
  const result = heavyComputation(1000000000); // 1M iterations
  res.json({ result });
});

// ❌ PAKE WORKER THREAD
import { Worker } from 'worker_threads';

app.get('/heavy', (req, res) => {
  const worker = new Worker('./heavy-worker.js');
  worker.on('message', (result) => {
    res.json({ result });
  });
});
```

❑ Sesi 2: Async Programming Deep Dive

2.1 Kenapa Async Penting?

Tanpa async, kode lo jalan **sequential** — satu per satu, nunggu yang sebelumnya selesai.

```
// ❌ Synchronous (blocking)
const data = fs.readFileSync('file.txt', 'utf-8');
// Baris ini NUNGGU file selesai dibaca
// Selagi nunggu, aplikasi lo BEKU
console.log(data);
```

Dengan async, sistem bisa kerja LAIN sambil nunggu:

```
// ❌ Async (non-blocking)
fs.readFile('file.txt', 'utf-8', (err, data) => {
  console.log(data);
});
// Baris ini LANGSUNG lanjut, ga nunggu file dibaca
// readFile dikirim ke libuv thread pool
// Pas selesai, callback masuk ke event loop
```

2.2 Callback → Promise → async/await (Evolusi)

Callback (sekarang udah jarang dipake langsung):

```

getUser(1, (err, user) => {
  if (err) return console.error(err);
  getPosts(user.id, (err, posts) => {
    if (err) return console.error(err);
    getComments(posts[0].id, (err, comments) => {
      if (err) return console.error(err);
      console.log(comments);
      // Bayangin kalo nested 5 level
      // Ini namanya CALLBACK HELL
    });
  });
});

```

Promise (nge-flat chain):

```

getUser(1)
  .then(user => getPosts(user.id))
  .then(posts => getComments(posts[0].id))
  .then(comments => console.log(comments))
  .catch(err => console.error(err));

```

async/await (baca kayak sync code):

```

async function main() {
  try {
    const user = await getUser(1);
    const posts = await getPosts(user.id);
    const comments = await getComments(posts[0].id);
    console.log(comments);
  } catch (err) {
    console.error(err);
  }
}

```

2.3 Promise Internals

Promise itu Cuma state machine dengan 3 state:

Pending → Fulfilled (resolve)

Pending → Rejected (reject)

Fulfilled → (udah final, ga bisa balik)

Rejected → (udah final)

Promise sederhana dari awal:

```

const myPromise = new Promise<string>((resolve, reject) => {
  // Ini executor – jalan SEGERA pas Promise dibuat
  setTimeout(() => {
    const success = true;

```

```

        if (success) {
            resolve("Selesai!");
        } else {
            reject(new Error("Gagal!"));
        }
    }, 1000);
});

myPromise
    .then(result => console.log(result)) // Microtask
    .catch(err => console.error(err))
    .finally(() => console.log("Always")); // Jalan apapun hasilnya

```

PENTING: Promise dibuat, executornya jalan SEKARANG juga (synchronous). Yang async cuma `.then()` callbacks.

```

console.log('1');
const p = new Promise(resolve => {
    console.log('2'); // ← Jalan sync di executor
    resolve('3');
});
p.then(console.log); // ← .then itu microtask, jalan belakangan
console.log('4');
// Output: 1, 2, 4, 3

```

2.4 async/await Under the Hood

`await` itu cuma syntactic sugar buat `Promise.then()`.

```

// Lo nulis:
async function fetchData() {
    const data = await fetch('/api/data');
    return data.json();
}

// Sebenarnya compiler ngubah jadi:
function fetchData() {
    return fetch('/api/data').then(data => data.json());
}

```

await menghentikan function, bukan thread:

```

async function getUsers() {
    const response = await fetch('/api/users');
    // □ Pas await, getUsers pause
    // Tapi event loop jalan terus
    // Thread bebas melayani request LAIN
}

```

```

    const data = await response.json();
    return data;
}

```

Error handling dengan async/await:

```

// ❌ Ini ga akan catch error yang di-throw:
async function broken() {
    throw new Error("Oops"); // Promise.reject()
    // Tapi ga ada yang nangkap
}
broken(); // Unhandled Promise rejection

// ✅ Pake .catch atau try/catch:
async function fixed() {
    try {
        await somethingRisky();
    } catch (err) {
        console.error("Ketangkep:", err);
    }
}

```

2.5 Promise Combinators

Promise.all — tunggu SEMUA selesai. Kalo 1 reject, semua gagal.

```

const [user, posts, comments] = await Promise.all([
    fetchUser(id),
    fetchPosts(id),
    fetchComments(id),
]);
// Semua jalan PARALEL
// Total waktu = paling lambat dari 3

```

Promise.allSettled — tunggu semua, catet hasil masing-masing.

```

const results = await Promise.allSettled([
    fetchPrimaryData(),
    fetchBackupData(), // Mungkin gagal
]);

results.forEach(r => {
    if (r.status === 'fulfilled') {
        console.log('Berhasil:', r.value);
    } else {
        console.log('Gagal:', r.reason);
    }
});

```

Promise.race — ambil yang paling cepet selesai.

```
const result = await Promise.race([
  fetchFromServer(),
  new Promise((_, reject) =>
    setTimeout(() => reject(new Error('Timeout!')), 5000)
  ),
]);
// Kalo server > 5 detik, timeout
```

Promise.any — ambil yang pertama BERHASIL.

```
const fastest = await Promise.any([
  fetchFromCDN(),
  fetchFromOrigin(),
  fetchFromBackup(),
]);
// Ambil yang paling cepet respon
```

2.6 Async Patterns

Sequential (satu-satu, total = A + B + C):

```
const a = await taskA(); // 2 detik
const b = await taskB(); // 2 detik
const c = await taskC(); // 2 detik
// Total: 6 detik
```

Parallel (barengan, total = max(A, B, C)):

```
const [a, b, c] = await Promise.all([
  taskA(), // 2 detik
  taskB(), // 2 detik
  taskC(), // 2 detik
]);
// Total: ~2 detik
```

Retry pattern (coba ulang kalo gagal):

```
async function fetchWithRetry(url: string, retries = 3): Promise<Response> {
  for (let i = 0; i < retries; i++) {
    try {
      return await fetch(url);
    } catch (err) {
      if (i === retries - 1) throw err;
      console.log(`Gagal, coba lagi (${i + 1}/${retries})...`);
      await new Promise(r => setTimeout(r, 1000 * Math.pow(2, i)));
      // Exponential backoff: 1s, 2s, 4s
    }
  }
}
```

```

    }
    throw new Error('Ga pernah berhasil');
}

```

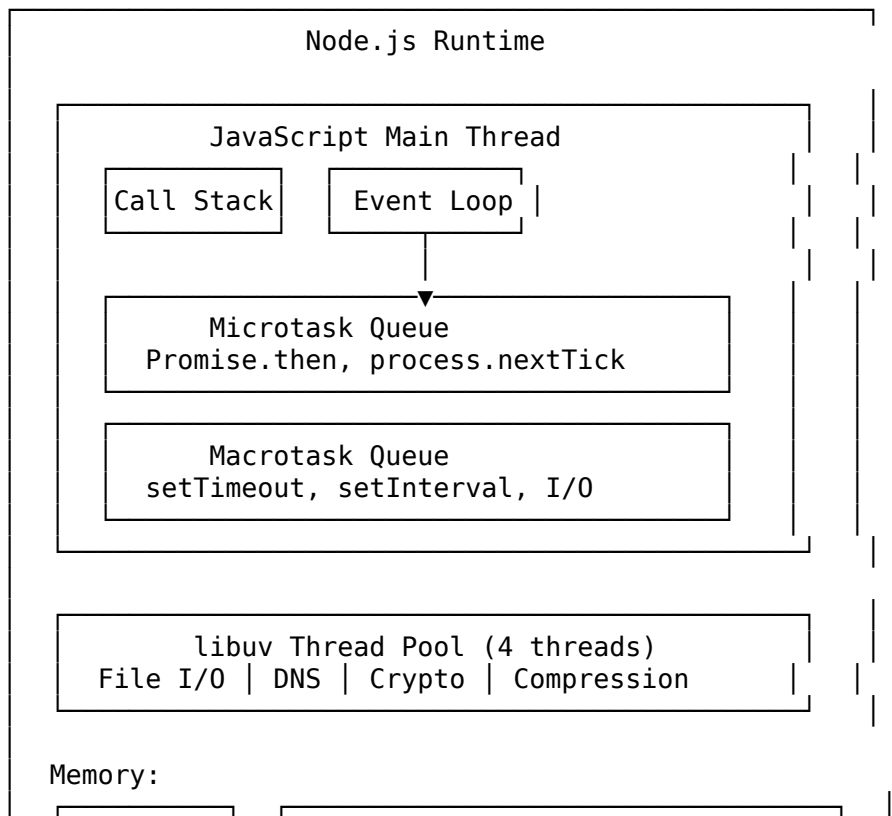
Async iterator (proses data stream):

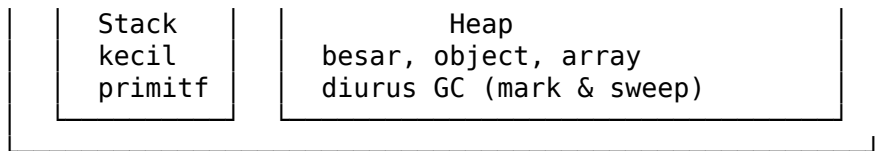
```

async function processInBatches(items: string[]) {
    const batchSize = 5;
    for (let i = 0; i < items.length; i += batchSize) {
        const batch = items.slice(i, i + batchSize);
        const results = await Promise.all(
            batch.map(item => processItem(item))
        );
        console.log(`Batch ${i / batchSize + 1} selesai`);
    }
}

```

□ Rangkuman Visual





□ Latihan

1. **Event loop prediction:** Tulis output dari kode ini:

```
console.log('a');  
setTimeout(() => console.log('b'), 0);  
Promise.resolve().then(() => console.log('c'));  
console.log('d');
```

2. **Memory leak detector:** Cari potential memory leak di kode class Chat di atas. Benerin.
3. **Async refactor:** Ubah callback hell jadi async/await (3 level nesting).
4. **Promise combinator:** Dari 4 API calls, jalankan paralel, tapi kalo salah satu gagal, tetap ambil yang berhasil.
5. **Heavy computation:** Simulasi CPU-bound task (loop 1M) di main thread. Ukur berapa lama block event loop. Trus pindahkan ke Worker Thread.